

Classifying patterns of marine particle distribution

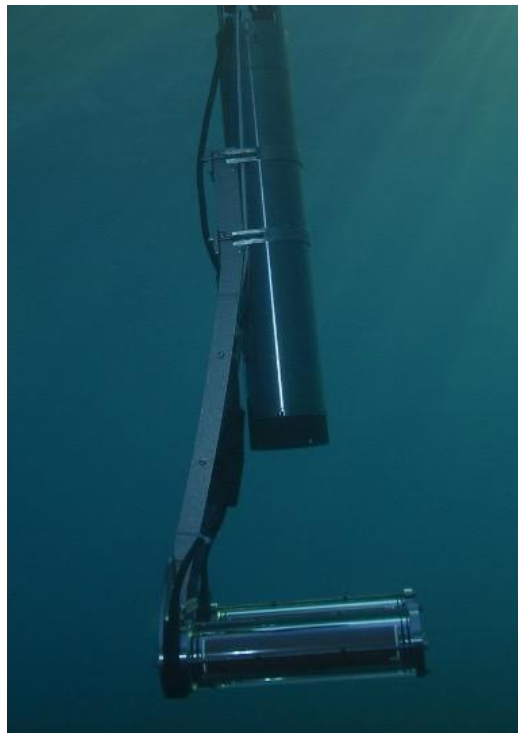
Group: Bianca Thobor, Sina Hallmann, Tobias Hölter, Sina Acksen

Course: Machine Learning with TensorFlow

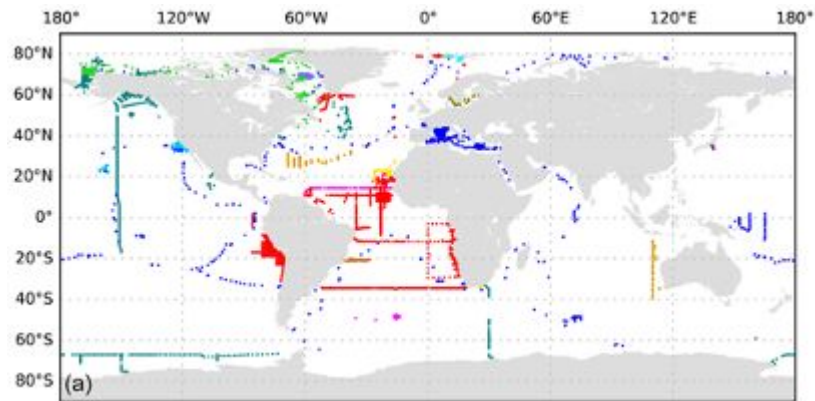
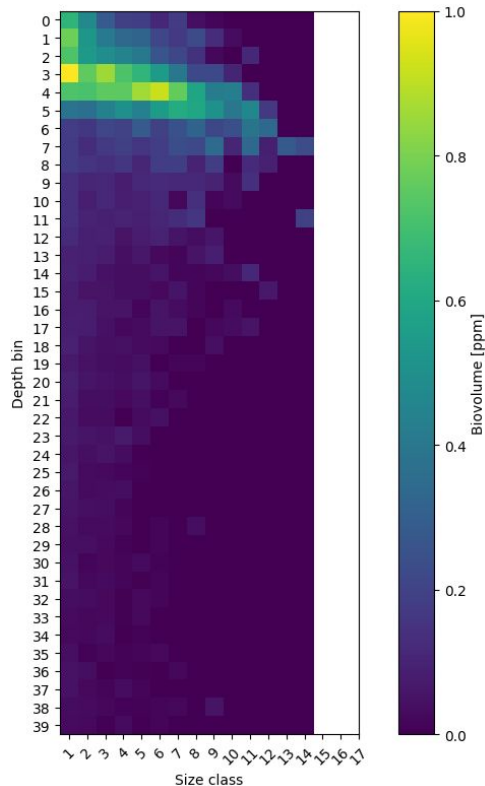
22.01.2025

2. Introduction

What environmental conditions cause carbon transfer to the deep ocean?

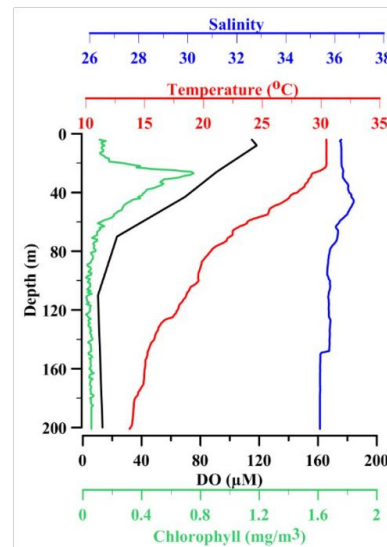


UVP camera



Kiko et al. 2022

Global dataset (> 8,000 profiles)



Model data

3. Literature Review: time-series heatmaps

From time series to image analysis: A transfer learning approach for night setback identification of district heating substations

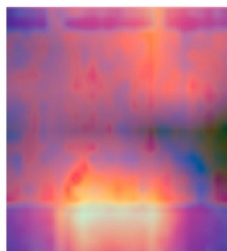
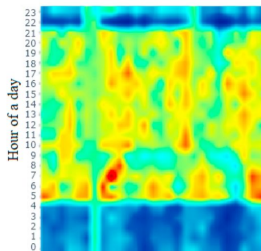
Fan Zhang^{a,b,*}, Chris Bales^b, Hasan Fleyeh^a

^a Department of Microdata Analysis, Dalarna University, Falun, 79188, Sweden

^b Department of Energy Technology, Dalarna University, Falun, 79188, Sweden

“...converts the original time series data into heatmap images, which **shifts the original research problem from conventional time series analysis into the image classification domain.**”

Resized
image



Highlighting
informative
regions

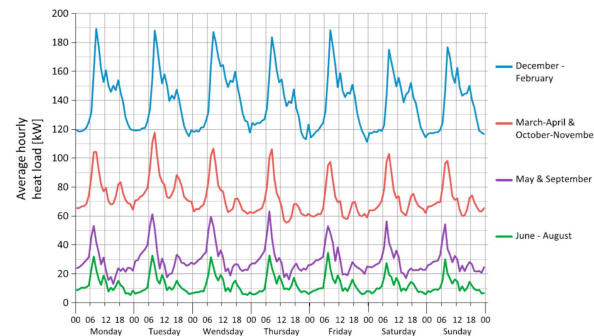
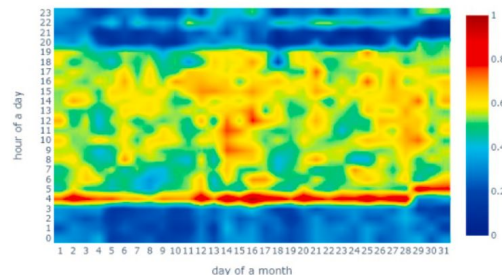


Fig. 1. Sample heat load patterns of night setback setting [10].

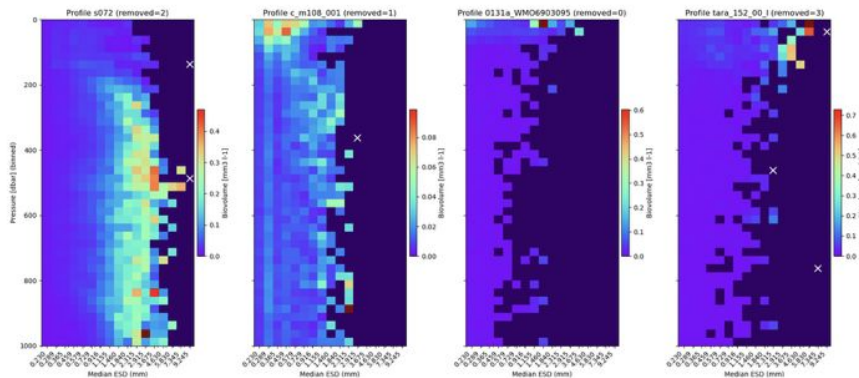


4. Dataset Characteristics

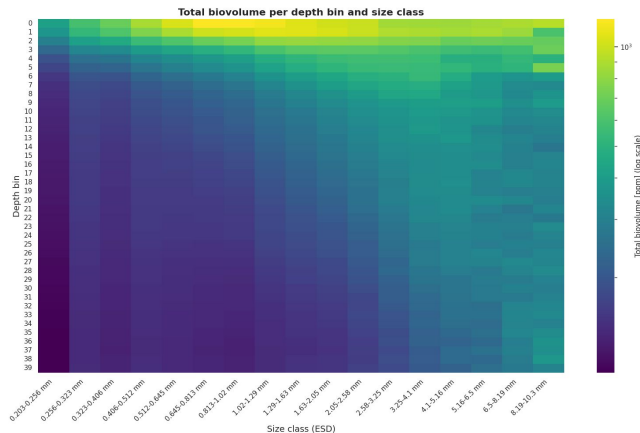
Features:

Heatmaps of particle distribution

5619 Profiles

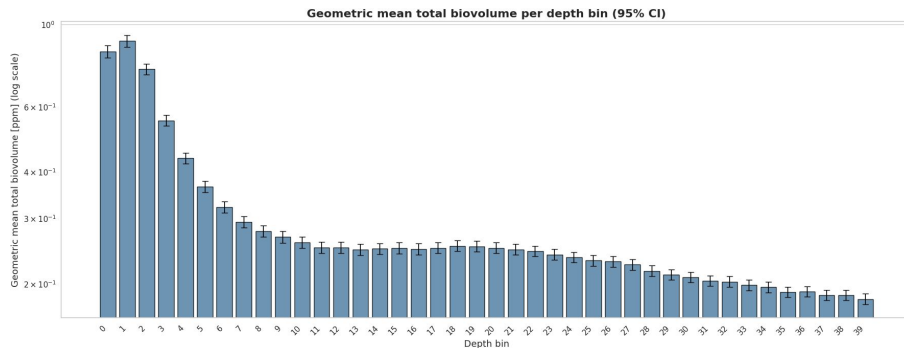


→ High correlations between biovolume sizes and depths



17 size classes (~ 0.2 - 10 mm)

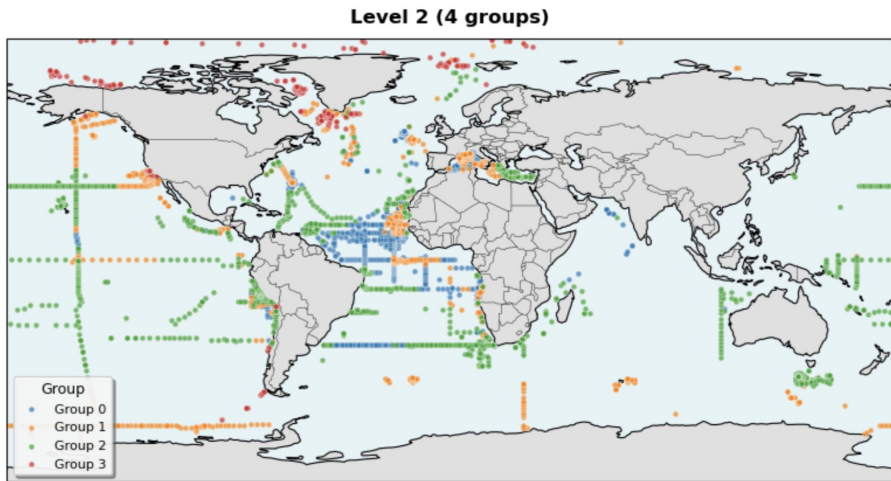
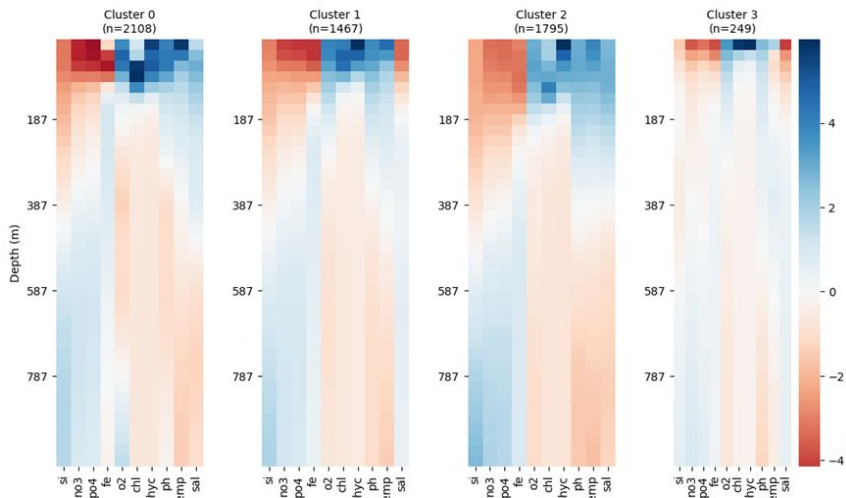
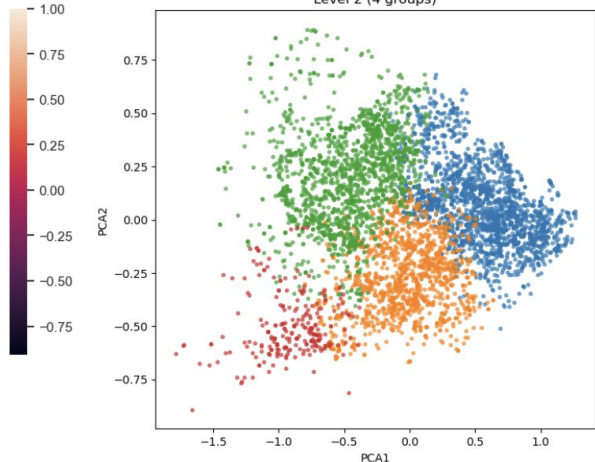
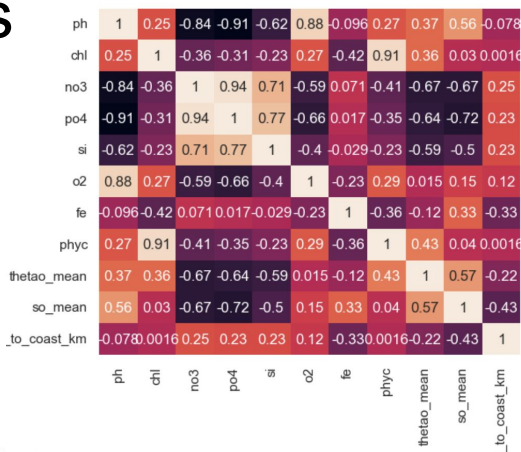
40 depth bins of 25 m (0-1000 m)



4. Dataset Characteristics

Labels:

4 Clusters of environmental conditions



5. Evaluation metrics & baseline Model

- Random Forest CV
- Ensemble Diversity
- Balanced Accuracy
- Surface & Size Signals
- Flattened Input
- Spatial Limitation

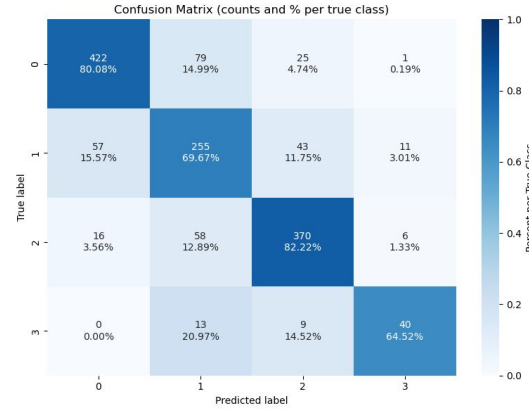


fig. 5.1

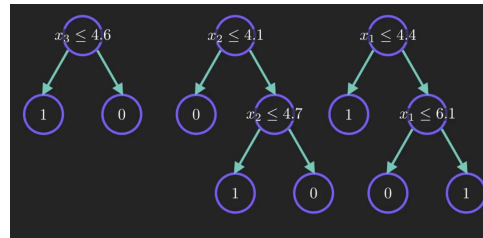


fig. 5.3

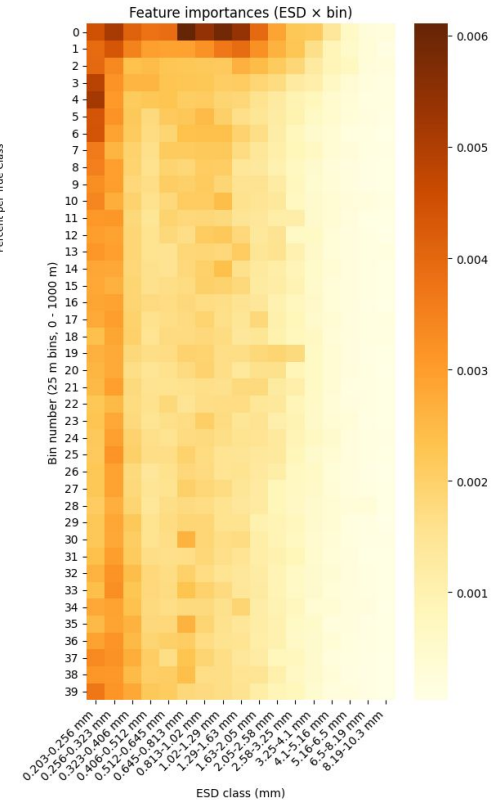
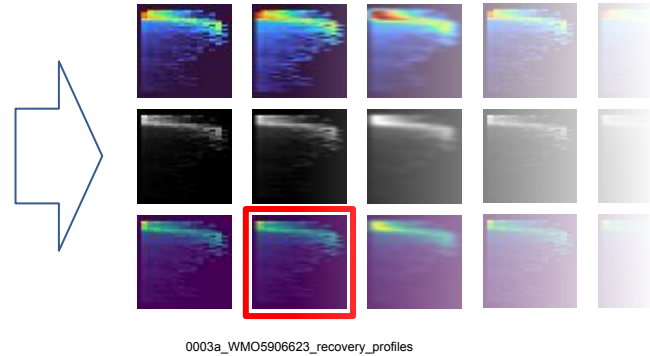
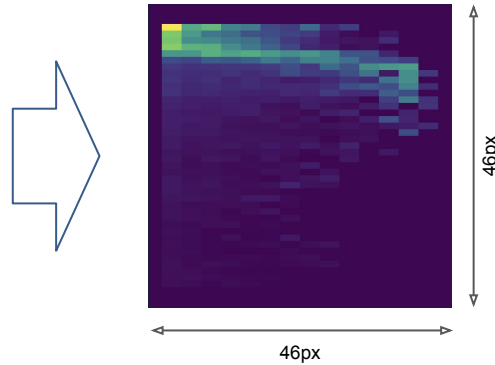
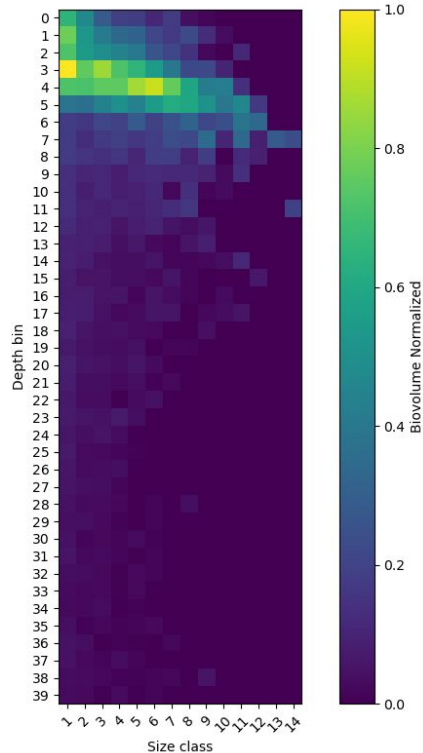


fig. 5.2

Feature engineering for CNN | data to image processing



0003a_WMO5906623_recovery_profiles

1. **Normalization** of Profile Data
2. Stripping **irrelevant Classes** (15,16,17) → to **14 Size Classes**
3. **Padding**: 40 rows (vertically) + **4 rows padding** (top/bottom) x 14 Size Classes + **1 column padding** (left/right)
4. **16 columns by 48 rows** → 3:1 ratio (when displayed in a square)
5. Creating 48x48 Pixel Square
6. **Cropping** → 46x46 for **3x3px kernel**
7. **Testing different themes and interpolations with CNN and comparing results before hyperparameter tuning and/or testing transfer models**

6. CNN model tuning



Config parameters:

- learning rate & reduction factor
- batch size
- dropout rate
- layer sizes
- noise level
- patience

Aim: improve balanced accuracy & reduce overfitting

```
def create_optuna_cnn_model(self, input_shape, config):
    """Create CNN model with Optuna-optimized hyperparameters"""

    print(f"Creating Optuna CNN model with input shape: {input_shape}")

    model = Sequential([
        # Input Layer with optimized noise augmentation
        BlockGaussianNoise(config["noise_level"], config["block_size"], seed=42, input_shape=input_shape),

        # First convolutional block with optimized filters
        Conv2D(config["conv_filters_1"], (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        Conv2D(config["conv_filters_1"], (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Dropout(config["dropout_rate_conv"]),

        # Second convolutional block with optimized filters
        Conv2D(config["conv_filters_2"], (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        Conv2D(config["conv_filters_2"], (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Dropout(config["dropout_rate_conv"]),

        # Third convolutional block with optimized filters
        Conv2D(config["conv_filters_3"], (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        Conv2D(config["conv_filters_3"], (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Dropout(config["dropout_rate_conv"]),

        # Fully connected layers with optimized sizes
        Flatten(),
        Dense(config["dense_units_1"], activation='relu'),
        BatchNormalization(),
        Dropout(config["dropout_rate_dense"]),

        Dense(config["dense_units_2"], activation='relu'),
        Dropout(config["dropout_rate_dense"]),

        Dense(self.n_classes, activation='softmax')
    ])

    return model
```

DETAILED CONFIGURATION FOR TOP MULTI-FOLD TRIALS:

Rank	Trial	LR	Batch	Conv Drop	Dense Drop	Conv Filters	Dense Units	Noise	Early Stop	Train/Val
1	17	1.69e-03	16	0.574	0.528	32-96-	96 384-	384 0.034	15	3148/1063

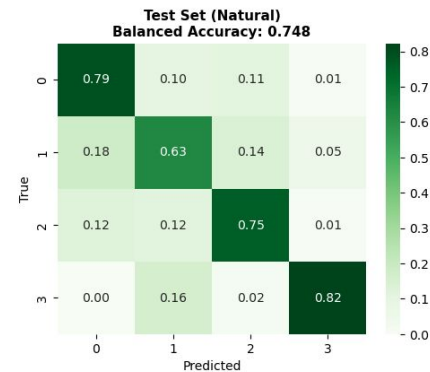
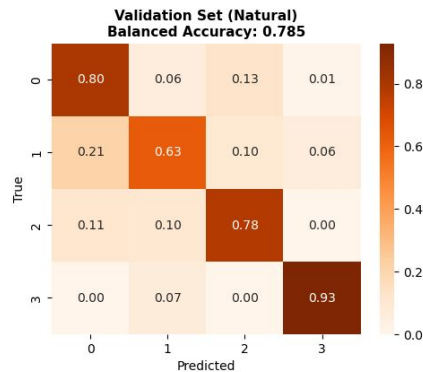
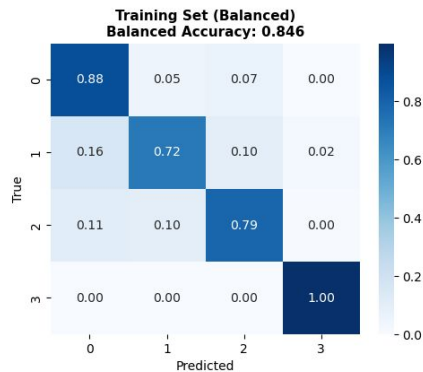
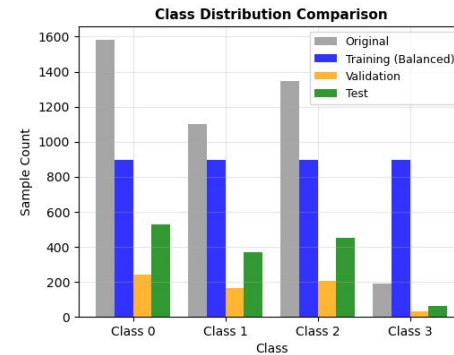
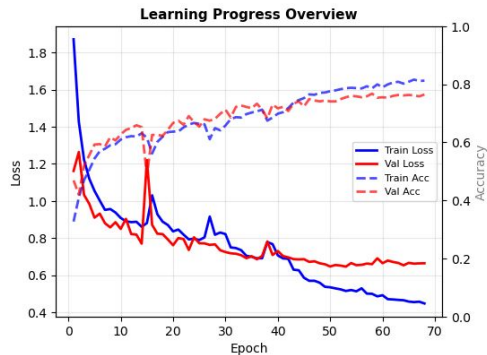
Final CNN performance

Balanced Acc.

Train: 0.85

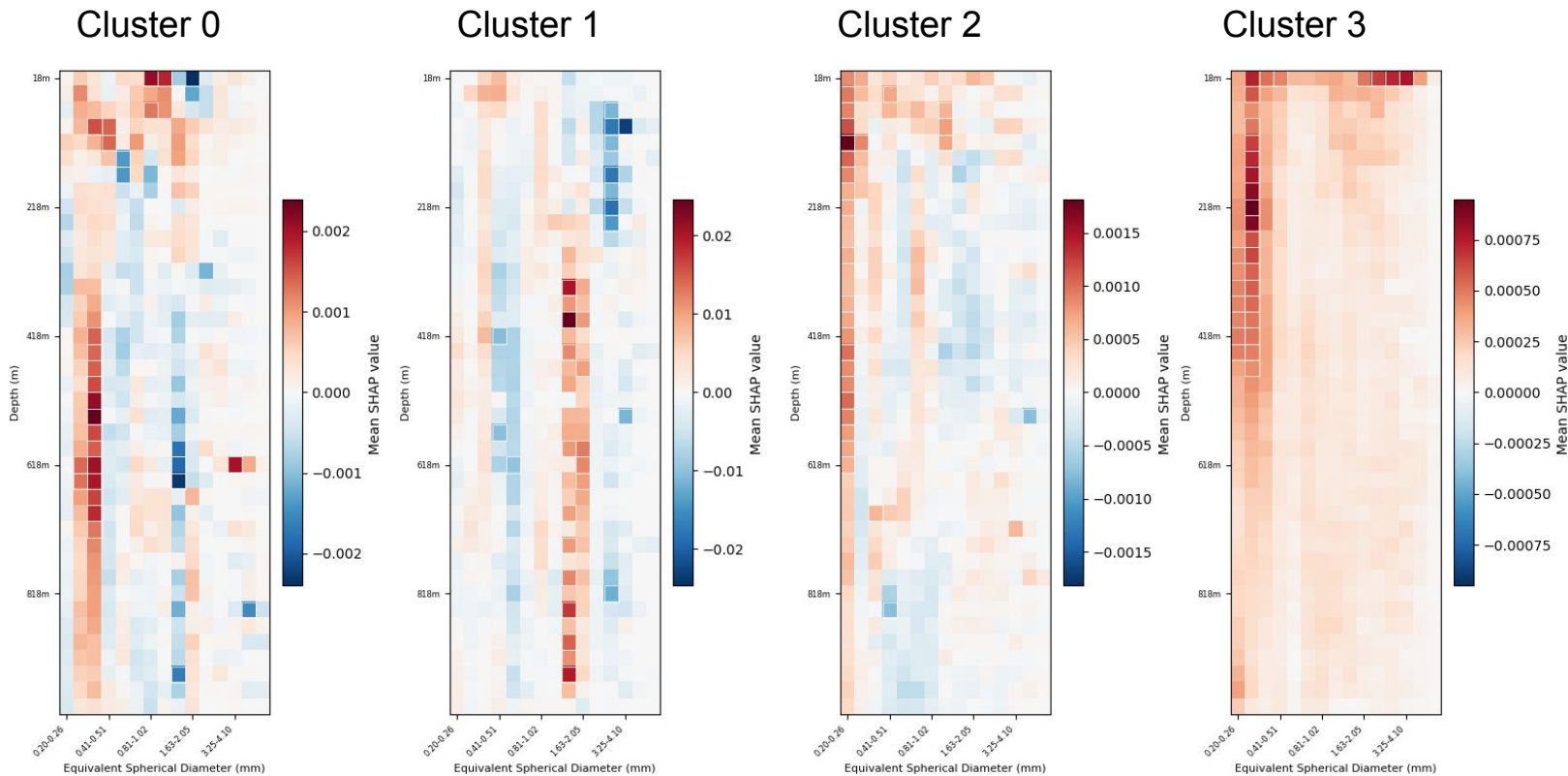
Val: 0.79

Test: 0.75

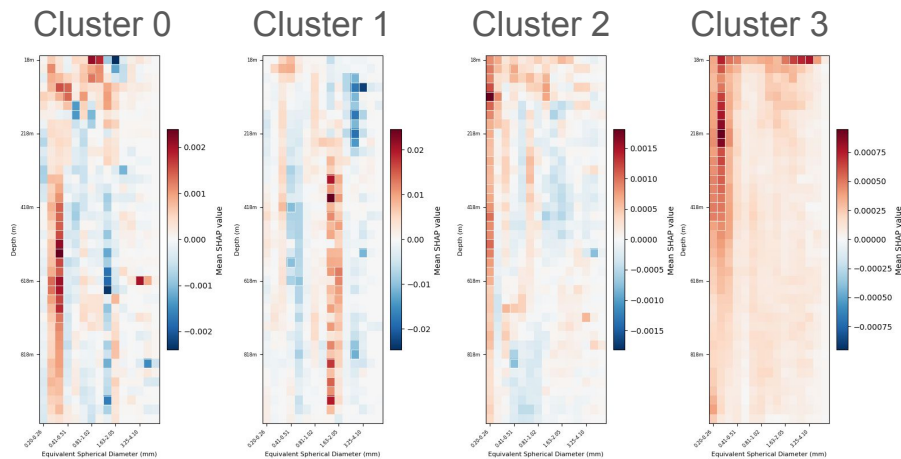


Mean feature contribution to clusters (SHAP)

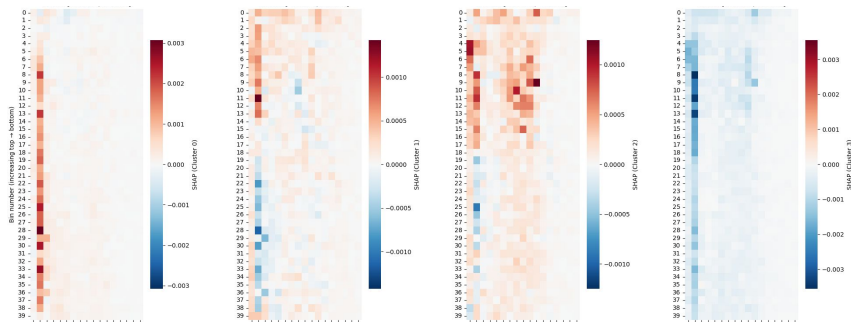
→ Which data are most important for predicting the cluster?



Mean feature contribution to clusters (SHAP)



SHAP analysis of CNN model



SHAP analysis of baseline model

7. Transfer learning

considered models

🔍 SonoNet, ResNet, EfficientNet-B3

→  EfficientNet-B3

variables

heatmap colour:

- default



- mp="gray"



number of unfrozen layers:

- 40
- 80
- 120
- 160
- 240

results

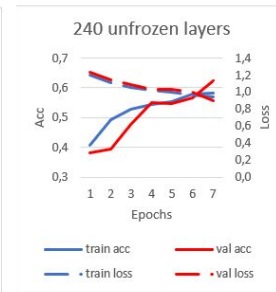
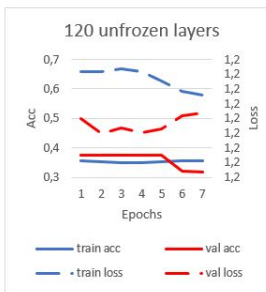
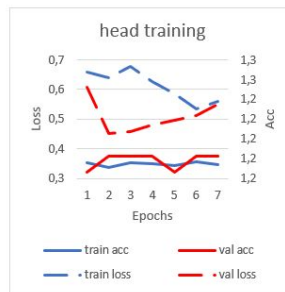
greyscale



default



unfrozen layers	train acc	train loss	val acc	val loss	train acc	train loss	val acc	val loss
0 (head)	0.36	1.24	0.37	1.22	0.35	1.24	0.37	1.24
40	0.36	1.23	0.37	1.21	0.36	1.23	0.39	1.23
80	0.37	1.22	0.40	1.26	0.36	1.23	0.32	1.23
120	0.43	1.18	0.54	1.09	0.36	1.23	0.32	1.23
160	0.49	1.10	0.60	0.96	0.46	1.15	0.49	1.12
200	0.52	1.05	0.46	1.07	0.49	1.10	0.38	1.31
240	0.58	0.95	0.60	0.95	0.58	0.94	0.62	0.90



→ best results with 240 unfrozen layers (val acc: 0.62)

Model performance table & discussion

number of features: $5619 \times 40 \times 17 = 3,816,840$

features per profile: 680

Model (best performing in Cross-Validation)	Trainable parameters	Training (Bal. Accuracy)	Test (Bal. Accuracy)	Time
Baseline model (Random Forest)	500 trees	0.99	0.74	0.03 min (2 s)
Simple CNN	1,359,588	0.85	0.75	7 min
Transfer learning model (EfficientNet-B3)	4,288,798	0.46	0.47	~ 300 min (5 h)

quality (Accuracy)

- comparable test results of Baseline model and simple CNN
- overfitting of Baseline model
- Transfer learning model doesn't reach quality of the other two models

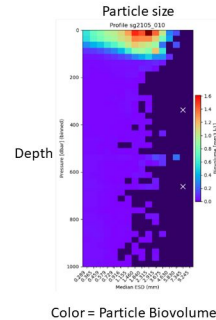
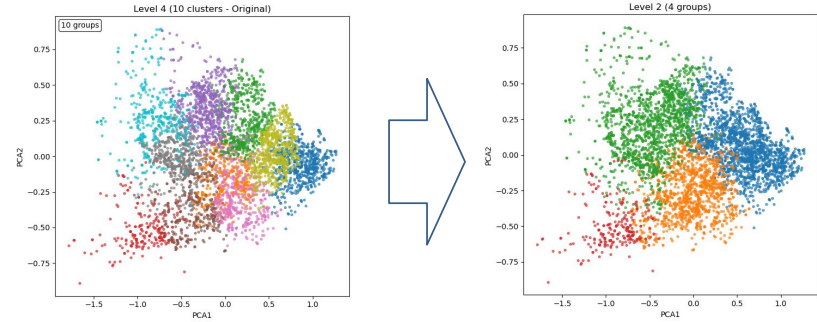
time

Baseline model is clearly superior to the other two models

8. Challenges and Errors

Cluster Reduction! We found out late, that we should reduce the number of classes to predict, due to overlapping environmental conditions.

Image Classification? Figuring out whether we wanted to do a classification or regression task. Also deciding to predict environm. regimes over bio volumes. (which way around?)



Aim: identify important features (depth / size) for each environmental cluster

Example environmental regimes/clusters:



„surface-stratified oligotrophic“



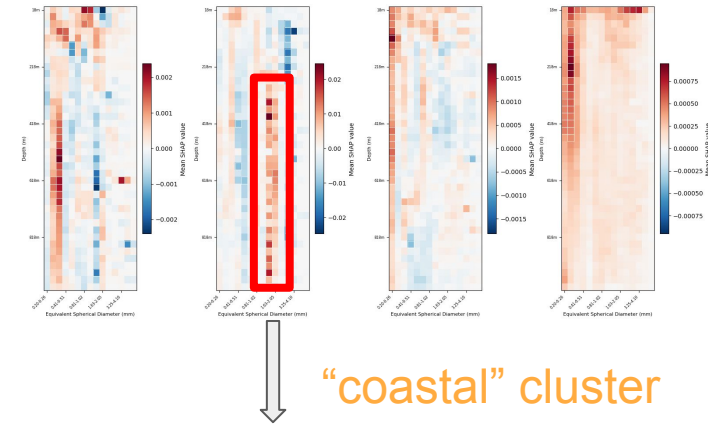
„subsurface chlorophyll maximum“



„Oxygen minimum zone“

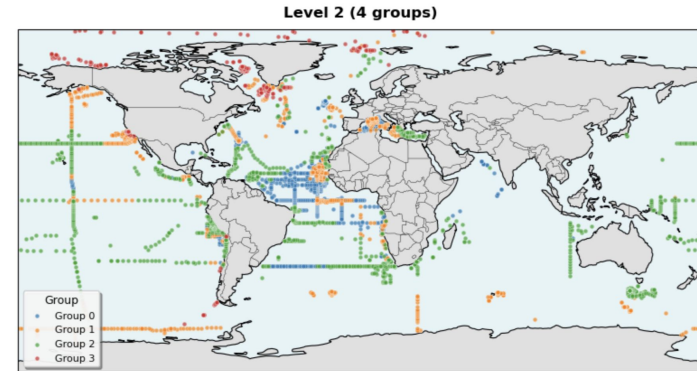
10. Conclusion and Future Work

- High variation in natural data: prediction accuracy of $\sim 75\%$ is good
- CNN-approach identifies meaningful patterns



Next steps:

- Look at individual oceans (e.g., Atlantic)
- Test different environmental clusters



Q & A